

プログラミングI 演習

Computer Programming I: Exercises

第13回 構造体

担当

A-P1 辻 愛里

A-P2 北 直樹

A-P3 清水 昭伸

本日の講義

本日の目標

- 構造体の利点を知ろう
- 構造体の宣言方法を覚えよう
- 構造体を利用したプログラムを作成してみよう

本日の資料

1. 前回のおさらい
2. 構造体
3. 演習課題

1. 前回のおさらい

- 乱数の利用
 - 乱数の発生が必要となるシミュレーション
 - 本演習では、**確率の計算**に利用
- C言語では・・・
 - 「`rand()`」というものがあつた。
 - 擬似的な乱数列に基づいて、0 ～ ある数までの整数(環境による)を算出
 - 剰余計算をすることで、範囲を指定した。
- 毎回同じ乱数列にならないようにするために
 - 「`srand(数値)`」で乱数の種を設定した。

補足：正確さを要求される数値計算には不適。
さらに進んだ乱数は後期



1. 前回のおさらい

演習12-1 乱数を使った確率計算②: サイコロの目

前ページの例を改良し、サイコロの目がでる確率を求める。

[1] 1つのサイコロを振った時、各目の出る確率

- 試行回数は何回でもよい。
- 各目が出た回数と各目がでる確率を求める
- 理論値: 各目とも $0.166666\cdots$

[2] 2つのサイコロの目の和に関する確率

- 各目の和(2~12)に対して確率を求める
- 理論値: **2と12**: $0.027777\cdots$, **3と11**: $0.055555\cdots$,
4と10: $0.083333\cdots$, **5と9**: $0.111111\cdots$
6と8: $0.138888\cdots$, **7**: $0.166666\cdots$

理論値との差が
試行回数により
どう変化するかも
調べてみよう

1. 前回のおさらい

演習12-1[1] 解答例

```
#include <stdio.h>
#include <stdlib.h>
#define KAISUU 1000000

int main(void)
{
    int i, result, dice[6]={0,0,0,0,0,0}; /* dice[0]が1, dice[1]が2, ... */
    srand(0);                               /* 乱数列の初期化 */

    for(i=1; i<=KAISUU; i++){
        result = rand() % 6;
        dice[result]++;
    }
    printf("1の回数は%6d回: 確率%lf¥n", dice[0], (double)dice[0]/KAISUU);
    printf("2の回数は%6d回: 確率%lf¥n", dice[1], (double)dice[1]/KAISUU);
    printf("3の回数は%6d回: 確率%lf¥n", dice[2], (double)dice[2]/KAISUU);
    printf("4の回数は%6d回: 確率%lf¥n", dice[3], (double)dice[3]/KAISUU);
    printf("5の回数は%6d回: 確率%lf¥n", dice[4], (double)dice[4]/KAISUU);
    printf("6の回数は%6d回: 確率%lf¥n", dice[5], (double)dice[5]/KAISUU);

    return 0;
}
```

```
1の回数は167024回: 確率0.167024
2の回数は167178回: 確率0.167178
3の回数は165916回: 確率0.165916
4の回数は166599回: 確率0.166599
5の回数は167051回: 確率0.167051
6の回数は166232回: 確率0.166232
```

実行結果.
理論値から「 $1/\sqrt{(\text{試行回数})}$ 」ぐらい
ズれることはよくある

1. 前回のおさらい

演習12-1[2] 解答例: 理論値からのズレも出してみた

```
#include <stdio.h>
#include <stdlib.h>
#define KAISUU 1000000
int main(void)
{
    int i, result1, result2, dice[11]={0,0,0,0,0,0,0,0,0,0,0};
    /* dice[0]が2, dice[1]が3, ..., dice[10]が12*/
    srand(0);

    for(i=1; i<=KAISUU; i++){
        result1 = rand() % 6;
        result2 = rand() % 6;
        dice[result1 + result2]++;
    }
    printf("和が 2の回数は%6d回: 確率%lf, 誤差%lf¥n", dice[0],
           (double)dice[0]/KAISUU, (double)dice[0]/KAISUU-(double)1/36);
    printf("和が 3の回数は%6d回: 確率%lf, 誤差%lf¥n", dice[1],
           (double)dice[1]/KAISUU, (double)dice[1]/KAISUU-(double)2/36);

    (和が4から12までは省略)

    return 0;
}
```

1. 前回のおさらい

演習12-1[2] 解答例: 理論値からのズレも出してみた

和が 2	の回数	は 27872回	: 確率0.027872,	誤差0.000094
和が 3	の回数	は 55848回	: 確率0.055848,	誤差0.000292
和が 4	の回数	は 83588回	: 確率0.083588,	誤差0.000255
和が 5	の回数	は 110607回	: 確率0.110607,	誤差-0.000504
和が 6	の回数	は 138804回	: 確率0.138804,	誤差-0.000085
和が 7	の回数	は 166552回	: 確率0.166552,	誤差-0.000115
和が 8	の回数	は 138700回	: 確率0.138700,	誤差-0.000189
和が 9	の回数	は 111255回	: 確率0.111255,	誤差0.000144
和が 10	の回数	は 83371回	: 確率0.083371,	誤差0.000038
和が 11	の回数	は 55614回	: 確率0.055614,	誤差0.000058
和が 12	の回数	は 27789回	: 確率0.027789,	誤差0.000011

理論値からおおよそ「 $1/\sqrt{(\text{試行回数})}$ 」のズレに収まっていることを確認しよう

1. 前回のおさらい

演習12-2 じゃんけんプログラム

基本方針:

- ① 「手」をユーザーから数字で入力してもらう
「1.ぐー、2.ちょき、3.ぱー」
- ② コンピュータの出す手を決める
(ここで, rand()を利用)
- ③ 勝敗の判定と結果の表示を行う

入力によって結果の変わる
プログラムを作成する

実行例

手を入力

(1.ぐー、2.ちょき、3.ぱー): 1

相手は「1」を出しました。

あいこなので次の手を入力してください。: 2

相手は「3」を出しました。

あなたの勝ちです。

1回勝負のプログラムができれば
「〇回勝負」として、〇回繰り返し、
勝敗数を表示してみよう

余裕があれば、コンピュータを強く
(あるいは弱く)してみよう。
(〇%の確率で勝つ(負ける)手を出
す, など)

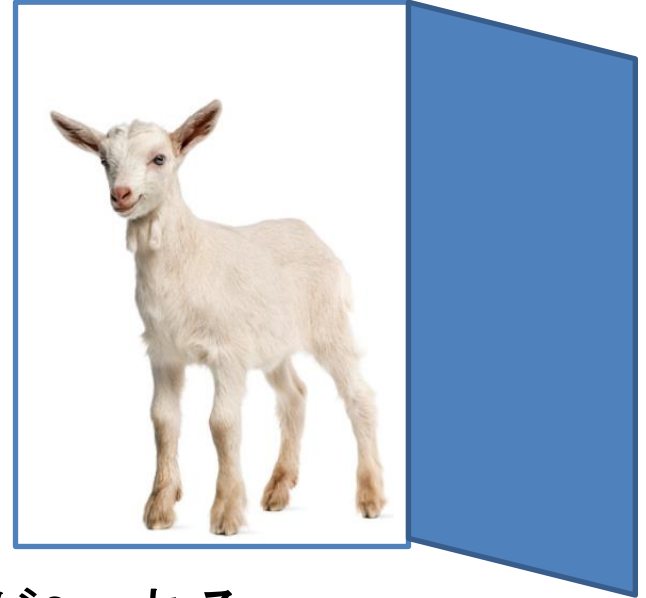
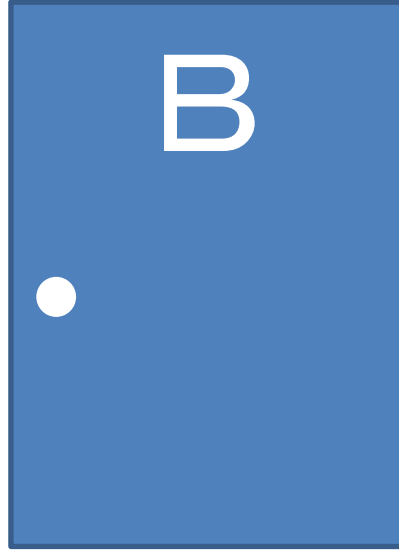
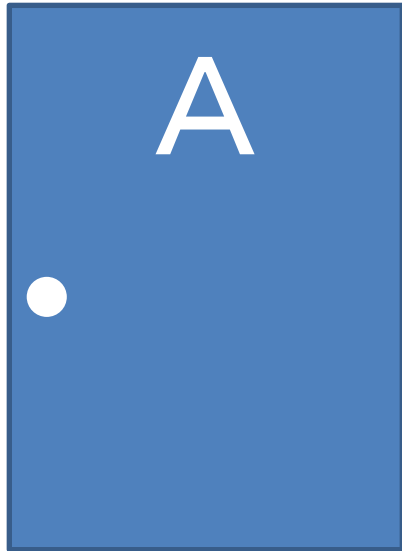
1. 前回のおさらい

演習12-2 解答例

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int User, CPU;          /* 出した手の番号を格納する変数 */
    int flag=0;             /* 勝敗が決まった否かの判定フラグ */
    srand(0);
    printf("手を入力(1.ぐー、2.ちょき、3.ぱー):");
    while(flag != 1) {
        scanf("%d",&User);          /* 手の入力 */
        CPU = rand()%3 + 1;          /* CPUの手を乱数で決定 */
        printf("相手は「%d」を出しました。¥n",CPU);          /* CPUの手を表示 */
        /* 勝敗判定 */
        if( (User==1 && CPU==2) || (User==2 && CPU==3) || (User==3 && CPU==1)) {
            printf("あなたの勝ちです。¥n");
            flag = 1;
        } else if(User == CPU) {
            printf("あいこなので次の手を入力してください。");
        } else {
            printf("あなたの負けです。¥n");
            flag = 1;
        }
    }
    return 0;
}
```

1. 前回のおさらい

発展12-1 モンティホール問題



- ・A～Cのドアがある. アタリが1つ, ハズレが2つある.
- ・あなたは, ドアを1つ選ぶ. (ここでは, ドアを開けない)
- ・その後, **あなたが選んでない「ハズレ」のドア**を司会者が開ける.
- ・その後, 司会者から「ドアを選び直しても良い」と言われる.
→ ドアを選び直し(変え)た方が良いでしょうか？

1. 前回のおさらい

発展12-1 解答例

プログラムは補足資料 (MontyHall.c) として配布します.

実行結果 (どんな状況でも選び直す (変えた) 場合)

```
成功回数は666829回: 確率0.666829  
失敗回数は333171回: 確率0.333171
```

実行結果 (どんな状況でも選び直さない (変えない) 場合)

```
成功回数は333171回: 確率0.333171  
失敗回数は666829回: 確率0.666829
```

2. 構造体

構造体とは

情報の中には、関連性の強い
グループのようなものがある

例:プロフィール(氏名、年齢、性別、血液型...)
日時(年、月、日、曜日、時、分、秒)
住所録(氏名、住所、電話番号...)



関連性の高いものは、まとまっていた
方が扱いやすい

変数をグループにまとめる機能が
あれば、プログラムが楽に(美しく)
なるのではないかな？



構造体

というものがある

2. 構造体

num → 分子 (numerator)
den → 分母 (denominator)

例：分数のかけ算をするプログラム

```
int main(void)
{
    int Xnum; //分数Xの分子
    int Xden; //分数Xの分母
    int Ynum; //分数Yの分子
    int Yden; //分数Yの分母
    int num; //答えの分子
    int den; //答えの分母

    //分数のかけ算
    num = Xnum * Ynum;
    den = Xden * Yden;
    return 0;
}
```

2. 構造体

例：分数のかけ算をするプログラム

```
int main(void)
{
    int Xnum; //分数Xの分子
    int Xden; //分数Xの分母
    int Ynum; //分数Yの分子
    int Yden; //分数Yの分母
    int num; //答えの分子
    int den; //答えの分母

    //分数のかけ算
    num = Xnum * Ynum;
    den = Xden*Yden;
    return 0;
}
```

分数は2つの数が必ずセットで利用されるのに、わざわざ別変数として扱うのは、混乱や間違いの元

2. 構造体

例：分数のかけ算をするプログラム

```
#define num 0
#define den 1
int main(void)
{
    int X[2]; //分数X
    int Y[2]; //分数Y
    int ans[2]; //答え

    //分数のかけ算
    ans[num] = X[num] * Y[num];
    ans[den] = X[den] * Y[den];
    return 0;
}
```

配列を使ったちょっとした工夫
ちょっとわかりやすくなった？
(人によっては前の方が・・・ということも)

2. 構造体

構造体

```
typedef struct  
{  
    int num;  
    int den;  
} Bunsuu;
```

```
int main(void)  
{  
    Bunsuu X; //分数X  
    Bunsuu Y; //分数Y  
    Bunsuu ans; //答え  
  
    //分数のかけ算  
    ans.num = X.num * Y.num;  
    ans.den = X.den * Y.den;  
    return 0;  
}
```

構造体(struct)の型定義

Bunsuuという新しい型(intやfloatの仲間)を作っている

プログラマが自由に変数を構造化して、新しい型を作ることができる

numやden(構造体の中の変数)をメンバとよぶ

2. 構造体

構造体

```
typedef struct  
{  
    int num;  
    int den;  
} Bunsuu;
```

```
int main(void)  
{
```

```
    Bunsuu X; //分数X  
    Bunsuu Y; //分数Y  
    Bunsuu ans; //答え
```

```
    //分数のかけ算
```

```
    ans.num = X.num * Y.num;  
    ans.den = X.den * Y.den;  
    return 0;
```

```
}
```

構造体の変数(実体)の宣言

int型やfloat型と同様に、
「Bunsuu型」の変数(X, Y, ans)
を作成している

2. 構造体

構造体

```
typedef struct
{
    int num;
    int den;
} Bunsuu;

int main(void)
{
    Bunsuu X; //分数X
    Bunsuu Y; //分数Y
    Bunsuu ans; //答え

    //分数のかけ算
    ans.num = X.num * Y.num;
    ans.den = X.den * Y.den;
    return 0;
}
```

構造体メンバの参照方法

変数名.メンバ名

というように記述する
(〇〇の(≡「.」)△ △ という感じ)

分数Xの分母=10ならば..
X.den = 10;

2. 構造体

構造体のコピー

```
typedef struct
{
    int num;
    int den;
} Bunsuu;

int main(void)
{
    Bunsuu X; //分数X
    Bunsuu Y; //分数Y
    Bunsuu ans; //答え

    //構造体のコピー
    X = Y;
    return 0;
}
```

構造体は通常の変数のように
代入によりコピーできる

```
struct
{
    char string[32];
};
```

という構造体の場合、メンバ
の文字列もコピーされる

補足. 構造体の定義(上級者向け)

定義には色々あるけれど...

```
struct  
{  
    int num;  
    int den;  
};
```

「名前のない」構造体(無名構造体)の定義
これだけでは、使いようがない・・・
→ 型がないので変数も定義できない

```
struct  
{  
    int num;  
    int den;  
} x,y;
```

「名前のない」構造体を型にもつ「変数」x,yを
同時に作ってあげる。

補足. 構造体の定義(上級者向け)

構造体の定義色々..

```
struct Bunsuu  
{  
    int num;  
    int den;  
};
```

「名前のある」構造体(Bunsuu)の定義

プログラム中でこの名前を使って変数を作ることができる

struct Bunsuu x,y;



C言語では、「後ろの名前は構造体です」というのを教えるために、前に「struct」と書く必要がある。

補足. 構造体の定義(上級者向け)

構造体の定義色々..

```
struct Bunsuu  
{  
    int num;  
    int den;  
};
```

「名前のある」構造体(Bunsuu)の定義

プログラム中でこの名前を使って変数を作ることができる

struct Bunsuu x,y;



C言語では、「後ろの名前は構造体です」というのを教えるために、前に「struct」と書く必要がある。

```
struct Bunsuu  
{  
    int num;  
    int den;  
}x,y;
```

その場で変数を作ること

補足. 構造体の定義(上級者向け)

構造体の定義色々..

```
typedef struct  
{  
    int num;  
    int den;  
} Bunsuu;
```

typedefを使う方法

```
typedef OO MyType;
```

OOという型を
MyTypeという名前の型として扱う

```
typedef int Kazu;  
Kazu a=10;
```

OOが「名前のない構造体」であっても
同じように宣言ができる

```
Bunsuu X;
```

「struct」をつけなくても平気になる
利点がある

ここから演習課題です

3. 演習課題

プログラムは演習10-4で示したもの

演習13-1 構造体の定義

```
#include<stdio.h>
int main(void){
    int i,j;
    char NameList[2][2][51];          /* 1つめは名前ID, 2つめは情報ID, 3つめは文字列入力用 */
    for(i=0; i<=1; i++){
        for(j=0; j<=1; j++){
            if(j==0){
                printf("ID: %dの名前を入力(全角25文字まで)", i);
                scanf("%s", NameList[i][j]);
            }else{
                printf("ID: %dの住所を入力(全角25文字まで)", i);
                scanf("%s", NameList[i][j]);
            }
        }
    }
    for(i=0; i<=1; i++){
        for(j=0; j<=1; j++){
            if(j==0){
                printf("ID: %dの名前は%s¥n", i, NameList[i][j]);
            }else{
                printf("ID: %dの住所は%s¥n", i, NameList[i][j]);
            }
        }
    }
    return 0;
}
```

以降のスライドの説明に従って構造体を定義し、実行してみよう。このプログラムと同じ結果が得られればOK

3. 演習課題

演習13-1 構造体の定義

1枚前のスライドのコードを説明のために簡素化
(※実際にはこの作業をする必要はありません)

```
#include<stdio.h>
int main(){
    int i,j;
    char NameList[2][2][51]; /* 1つめは名前ID, 2つめは情報ID, 3つめは文字列入力用 */
    for(i=0; i<=1; i++)
        for(j=0; j<=1; j++)
            if(j==0){ printf("ID: %dの名前:", i); scanf("%s", NameList[i][j]); }
            else      { printf("ID: %dの住所:", i); scanf("%s", NameList[i][j]); }
    for(i=0; i<=1; i++)
        for(j=0; j<=1; j++)
            if(j==0) printf("ID: %dの名前は%s¥n", i, NameList[i][j]);
            else      printf("ID: %dの住所は%s¥n", i, NameList[i][j]);
    return 0;
}
```

3. 演習課題

演習13-1 構造体の定義 → ②構造体「AddrBook」の定義

```
#include<stdio.h>

typedef struct{
    char name[51];
    char address[51];
}AddrBook;

int main(){
    int i,j;
    char NameList[2][2][51]; /* 1つめは名前ID, 2つめは情報ID, 3つめは文字列入力用 */
    for(i=0; i<=1; i++)
        for(j=0; j<=1; j++)
            if(j==0){ printf("ID: %dの名前:", i); scanf("%s", NameList[i][j]); }
            else      { printf("ID: %dの住所:", i); scanf("%s", NameList[i][j]); }
    for(i=0; i<=1; i++)
        for(j=0; j<=1; j++)
            if(j==0) printf("ID: %dの名前は%s¥n", i, NameList[i][j]);
            else      printf("ID: %dの住所は%s¥n", i, NameList[i][j]);
    return 0;
}
```

3. 演習課題

演習13-1 構造体の定義 → ③AddrBook型の変数定義

```
#include<stdio.h>

typedef struct{
    char name[51];
    char address[51];
}AddrBook;

int main(){
    int i,j;
    AddrBook list[2];
    char NameList[2][2][51]; /* 1つめは名前ID, 2つめは情報ID, 3つめは文字列入力用 */
    for(i=0; i<=1; i++)
        for(j=0; j<=1; j++)
            if(j==0){ printf("ID: %dの名前:", i); scanf("%s", NameList[i][j]); }
            else      { printf("ID: %dの住所:", i); scanf("%s", NameList[i][j]); }
    for(i=0; i<=1; i++)
        for(j=0; j<=1; j++)
            if(j==0) printf("ID: %dの名前は%s¥n", i, NameList[i][j]);
            else      printf("ID: %dの住所は%s¥n", i, NameList[i][j]);
    return 0;
}
```

このように新しい型を使って、配列の定義もできます。

3. 演習課題

演習13-1 構造体の定義 → ④AddrBook型変数を利用する

```
#include<stdio.h>

typedef struct{
    char name[51];
    char address[51];
}AddrBook;

int main(){
    int i,j;
    AddrBook list[2];
char NameList[2][2][51]; /* 1つめは名前ID, 2つめは情報ID, 3つめは文字列入力用 */
    for(i=0; i<=1; i++)
        for(j=0; j<=1; j++)
            if(j==0){ printf("ID: %dの名前:", i); scanf("%s", list[i].name); }
            else      { printf("ID: %dの住所:", i); scanf("%s", list[i].address); }
    for(i=0; i<=1; i++)
        for(j=0; j<=1; j++)
            if(j==0) printf("ID: %dの名前は%s¥n", i, list[i].name);
            else      printf("ID: %dの住所は%s¥n", i, list[i].address);
    return 0;
}
```

ここまでできたら, NameListの宣言は不要

3. 演習課題

演習13-2 5科目 × 5人分の成績処理

5人分の国語, 数学, 理科, 社会, 英語(各100点満点)の点数を入力し, 各人の合計点・平均点, 各教科の平均点を出力するプログラムを作成せよ. ただし, 以下の構造体を利用すること. (変数名は変更しても良い)

```
typedef struct {  
    int Jpn; /* 国語 */  
    int Mat; /* 数学 */  
    int Sci; /* 理科 */  
    int Soc; /* 社会 */  
    int Eng; /* 英語 */  
} Student;
```

3. 演習課題

演習13-2 5科目×5人分の成績処理

ヒント: 構造体の定義 + 値の入出力 (まずはこのとおり実行してみよう)

```
#include<stdio.h>
#include<stdlib.h>
typedef struct{
    int Jpn; int Mat; int Sci; int Soc; int Eng;
}Student;

int main(){
    int i,j;
    Student s[5];

    /* 値の入力 */
    for(i=0; i<5; i++){
        s[i].Jpn = rand()%101; s[i].Mat = rand()%101; s[i].Sci = rand()%101;
        s[i].Soc = rand()%101; s[i].Eng = rand()%101;
    }
    /* 値の表示 */
    printf("学生 国語 数学 理科 社会 英語\n");
    for(i=0; i<5; i++){
        printf("s[%d]", i);
        printf("%5d", s[i].Jpn); printf("%5d", s[i].Mat);
        printf("%5d", s[i].Sci); printf("%5d", s[i].Soc);
        printf("%5d\n", s[i].Eng);
    }
    return 0;
}
```

3. 演習課題

発展13-1 30人分の成績処理＋ファイル入出力

補足資料「Result.txt」に30人分の漢字氏名, フリガナ, 各教科の得点が記入されている. 各教科の平均点, 各人の合計・平均・偏差値を出力できるプログラムを作成せよ. (出力先はコンソール上でもファイルでも良い)

ファイルの形式

氏名01\tフリガナ01\t国語01\t数学01\t理科01\t社会01\t英語01
氏名02\tフリガナ02\t国語02\t数学02\t理科02\t社会02\t英語02
....
氏名30\tフリガナ30\t国語30\t数学30\t理科30\t社会30\t英語30

30人分ある

サンプル「Result.txt」の点数は架空のものです.
実在の人物とは無関係です.

3. 演習課題

発展13-1 30人分の成績処理＋ファイル入出力

ファイルの形式

氏名01¥tフリガナ01¥t国語01¥t数学01¥t理科01¥t社会01¥t英語01
氏名02¥tフリガナ02¥t国語02¥t数学02¥t理科02¥t社会02¥t英語02
....
氏名30¥tフリガナ30¥t国語30¥t数学30¥t理科30¥t社会30¥t英語30

30人分ある

構造体の例

```
typedef struct
{
    char Name[32];
    char Kana[32];
    int  Score[5];
} Student;
```

ここに各教科の
得点を入れる

サンプル「Result.txt」の点数は架空のものです。
実在の人物とは無関係です。

3. 演習課題

発展13-1 30人分の成績処理＋ファイル入出力

```
int main(void){  
    Student ExamData[STUDENT_NUM];  
    int ID;  
  
    ReadFile(ExamData);  
  
    /* 点数の処理 */  
  
    return 0;  
}
```

構造体の配列を作成
「#define STUDENT_NUM 30」

ファイルからデータを
読み込む関数を作成

3. 演習課題

発展13-1 30人分の成績処理＋ファイル入出力

```
#include <stdio.h>
#define STUDENT_NUM 30

typedef struct { (略) } Student;

void ReadFile(Student StudentList[])
{
    int ID;
    FILE *fp = fopen("Result.txt","r");
    for(ID = 0 ;ID < STUDENT_NUM; ID++)
    {
        fscanf(fp,"%s¥t%s¥t%d¥t%d¥t%d¥t%d¥t%d",StudentList[ID].Name
        ,StudentList[ID].Kana
        ,&StudentList[ID].Score[0]
        ,&StudentList[ID].Score[1]
        ,&StudentList[ID].Score[2]
        ,&StudentList[ID].Score[3]
        ,&StudentList[ID].Score[4]);

    }
    fclose(fp);
}
```

データを格納するそれぞれのメンバ変数を渡す

タブが入っている場合でも、形式を指定することで、入力可能

fscanf(fp,"%s¥t%s¥t%d¥t%d¥t%d¥t%d¥t%d",StudentList[ID].Name
,StudentList[ID].Kana
,&StudentList[ID].Score[0]
,&StudentList[ID].Score[1]
,&StudentList[ID].Score[2]
,&StudentList[ID].Score[3]
,&StudentList[ID].Score[4]);

本日の説明内容はおしまいです。
課題に取り組んでください。